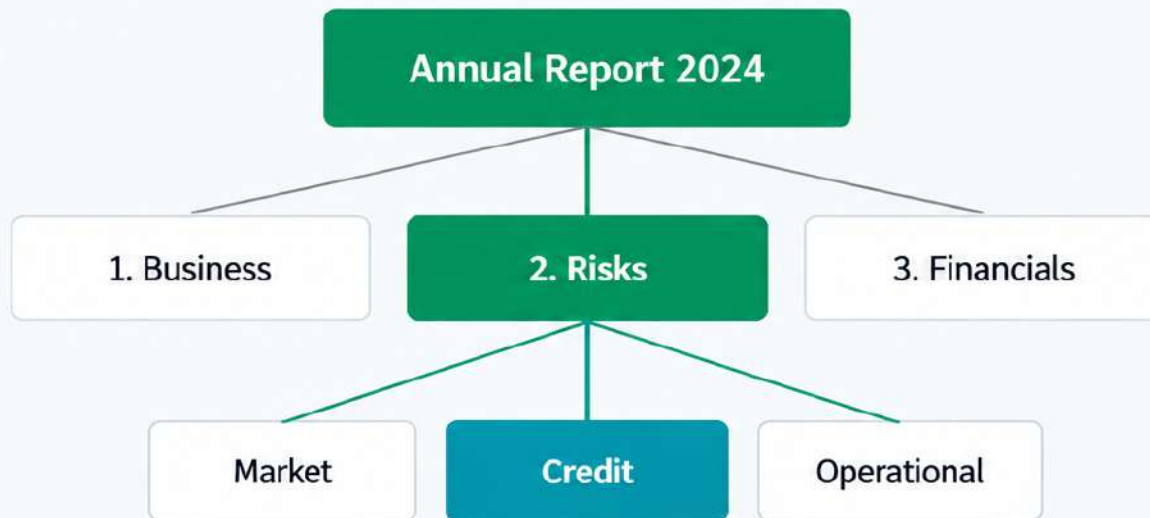


Vectorless RAG: Reasoning Through Structure

Skip embeddings entirely — let the LLM navigate the document like a human would.

HIERARCHICAL INDEX



LLM navigates: root → chapter → section → answer

Each node has a summary the LLM reads to decide where to go.

How the LLM navigates

1

Build the tree

Parse doc structure (headings, sections); generate summaries at each node — done once, offline.

2

LLM reads root summary

“Which chapter is most likely to contain the answer?”

3

Descend the tree

Repeat at each level until reaching a leaf section.

4

Read full section

No chunking — full context preserved.

5

Answer + cite path

Returns answer AND the navigation path.

Traditional RAG: The Real Picture

It's powerful — but it has known failure modes



STRENGTHS

Massive scale

Millions of docs, millisecond lookups

Mature ecosystem

Chroma, FAISS, Pinecone, Qdrant, Weaviate

Cheap retrieval

One embedding + ANN search per query

Great for factoids

Short, lookup-style questions

Domain agnostic

Works on any text — blogs, tickets, PDFs



WEAKNESSES

Chunking destroys context

"As defined in Section 3.2..." — meaningless when retrieved alone

Similarity ≠ relevance

Embeddings can match wrong things confidently

No cross-section reasoning

Can't answer "compare risks vs mitigations"

Hard to explain

Why was this chunk picked? Cosine score isn't enough

Embedding drift

Model changes = re-embed everything

Vectorless RAG: The Real Picture

Different tradeoffs — better for some workloads, worse for others



STRENGTHS

Preserves document context

Sections stay whole — no broken references

Cross-section reasoning

LLM can compare, contrast, synthesize

Explainable retrieval

Returns the navigation path, not a cosine score

No embedding pipeline

Skip the embed/index/refresh complexity

Plays well with structure

Reports, contracts, filings, textbooks shine



WEAKNESSES

Higher per-query cost

Multiple LLM calls to traverse the tree

Higher latency

Several hundred ms to a few seconds per query

Doesn't scale to millions

Works for tens to thousands of docs, not internet-scale

Needs structured docs

Random blog posts? Tree adds little value

Less mature tooling

PageIndex and a few others — ecosystem is younger

Use Traditional RAG when...



Massive heterogeneous corpora

Millions of mixed-format docs — blogs, tickets, transcripts, knowledge base articles.



Latency-critical apps

Chatbots, search-as-you-type, voice assistants. Every millisecond counts.



Short factoid queries

"What's the warranty period?" "Who's the CEO?"
The answer lives in one chunk.



Cost-sensitive at scale

Thousands of queries per day where every penny counts; LLM-tree-walks would be more expensive.

Use Vectorless RAG when...



Long, structured documents

Annual reports, 10-Ks, legal contracts, regulatory filings, research papers, textbooks.



Reasoning > similarity

“Compare risk factors in section 7 to mitigations in section 12.” Pure similarity can’t do this.



Explainability is required

Compliance, audit, legal, financial advisory. Show your work — not just the answer.



Chunking destroys context

Cross-references like “as defined earlier...” lose meaning when retrieved in isolation.

Side-by-Side: The Honest Comparison

Eight dimensions that matter in production systems

Dimension	Traditional RAG	Vectorless RAG
Scale	Millions of docs ✓	Tens to thousands
Latency / query	Milliseconds ✓	Hundreds of ms — seconds
Cost / query	Cheap ✓ (1 embed lookup)	Higher (multiple LLM calls)
Cross-section reasoning	Weak	Strong ✓
Explainability	Cosine score (opaque)	Navigation path ✓
Best for	Factoid Q&A, mixed corpora	Long structured docs
Setup complexity	Embedding pipeline + DB	Tree builder, no DB
Ecosystem maturity	Very mature ✓	Emerging

Key Takeaways

Five things to remember from this video

1

Traditional RAG = scale + speed

Vector similarity is unbeatable for millions of docs and millisecond lookups.

2

Vectorless RAG = reasoning + structure

LLM-driven tree navigation preserves context and explains itself.

3

They're not competitors

They're complementary. Pure vector search and pure tree navigation are both extremes.

4

The right pick depends on the doc, not the hype

Long structured filings → vectorless. Mixed knowledge base → vector. Big system → hybrid.

5

Production systems are going hybrid

Vectors narrow the search space; trees do the precise reasoning inside it.